AIML COURSE PROJECT · VIVA PREPARATION

Vani-Setu Viva Guide

Who builds what, the AI concepts behind each part, and answers to the questions examiners are likely to ask

Member	Roll number	AI area owned
Abdullah Naseem	S24CSEU1193	Letter recognition: landmarks, feature engineering, SVM, confidence, smoothing
Bhanu Satwik	S24CSEU0898	Words and phrases: neural network (MLP), two-hand ISL, LSTM for motion, smart word prediction
Manoj Kumar K	S24CSEU1239	Data and evaluation: dataset, augmentation, validation, model comparison, metrics, testing

Project guide	Dr. Mohd Anas
Department / Institute	School of Computer Science Engineering and Technology (CSET), Bennett University
Project	Vani-Setu: real-time sign language translation with hand landmarks and machine learning

How to use this guide. Each member owns one stage of the AI pipeline from start to finish, so everyone can answer technical ML questions about their own part. Read your own section fully, read Section 2 (everyone must know it), and read the one-minute scripts of the other two members.

Status tags. BUILT = in the code on the *improvements* branch and can be demonstrated. NEEDS DATA = the code is built but needs samples you record before it does anything. ADDING = still to be built. In the viva, say "we are implementing" for ADDING items and "built, waiting for our recordings" for NEEDS DATA items.

Contents

1. Work Division at a Glance 3

1.1 Contributions in detail . . . 3

1.2 Who presents what in the demo . . . 4

2. What Every Member Must Know 4

2.1 The 30-second pitch . . . 4

2.2 Numbers to remember . . . 4

2.3 What is ours and what we reused . . . 5

2.4 Questions any member can be asked . . . 5

3. Abdullah Naseem: Letter Recognition 6

3.1 Contribution . . . 6

3.2 Concepts to explain . . . 6

3.3 One-minute viva script . . . 8

3.4 Likely questions and answers . . . 8

4. Bhanu Satwik: Words, Phrases and Language 10

4.1 Contribution . . . 10

4.2 Concepts to explain . . . 10

4.3 One-minute viva script . . . 12

4.4 Likely questions and answers . . . 13

5. Manoj Kumar K: Data and Evaluation 14

5.1 Contribution . . . 14

5.2 Concepts to explain . . . 14

5.3 One-minute viva script . . . 16

5.4 Likely questions and answers . . . 16

6. Viva Day Checklist 17

1. Work Division at a Glance

The AI pipeline has three stages, and each member owns one. Abdullah turns a hand into a letter. Bhanu turns hands and letters into words, phrases and sentences. Manoj supplies the data and proves how well every model works. Each member also owns the app screens and APIs around their stage.

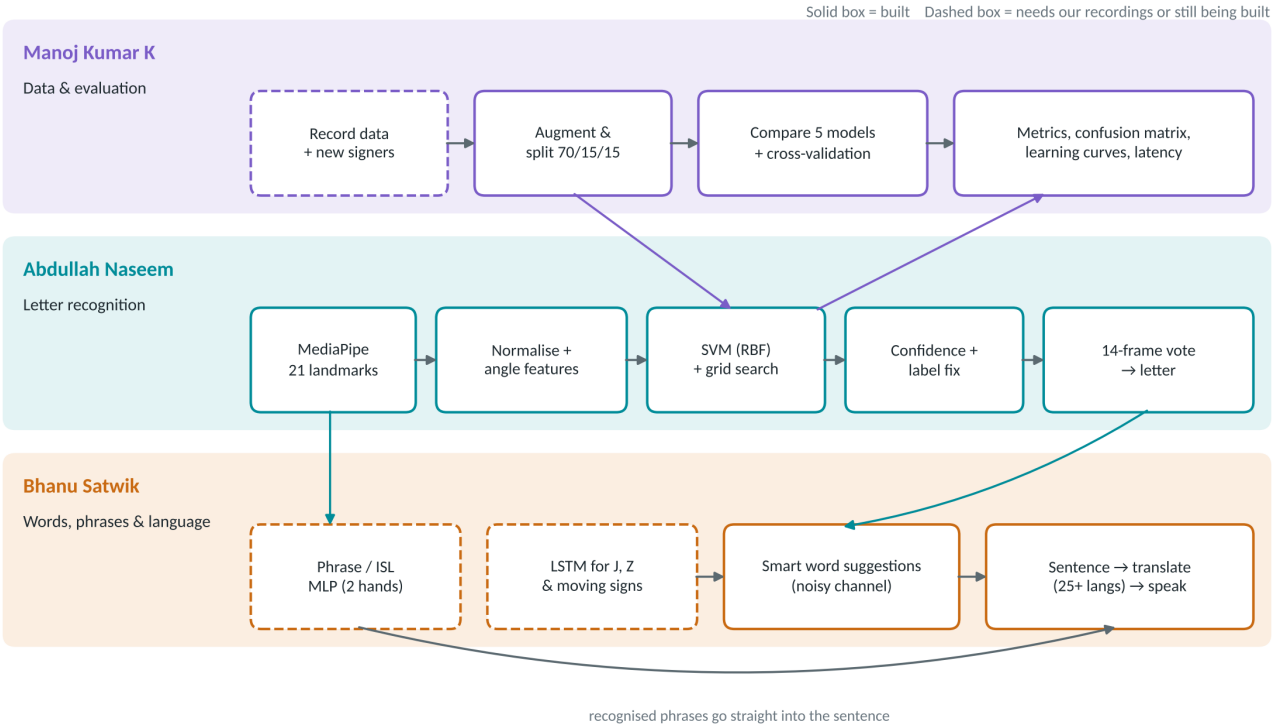


Figure 1. Ownership of the pipeline. Each lane is one member; arrows show how their work connects.

1.1 Contributions in detail

Member	AI / ML contribution	App and backend contribution	Files owned
Abdullah Naseem	Landmark normalisation, angle and distance features, SVM training and tuning, probability confidence, temporal smoothing	Camera screen, live classification loop, confidence bar	`features.py`, `train_classifier.py`, `/api/classify`
Bhanu Satwik	MLP neural network for phrases and ISL, two-hand features, LSTM for J, Z and moving signs, noisy-channel word prediction	Login, Phrase / ISL screens, Sign Guide, translation and speech	`nn/train_mlp.py`, `nn/numpy_mlp.py`, `suggest.py`, `signs.py`
Manoj Kumar K	Data recording, augmentation, 70/15/15 split, cross-validation, model comparison, metrics, learning curves, new-signer testing	Record and Retrain screens, practice mode	`data.py`, `augment.py`, `evaluate.py`, `compare_models.py`

1.2 Who presents what in the demo

Order	Presenter	Shows	Time
1	Abdullah	Problem, architecture, Spell mode with the confidence bar and the 14-frame vote	~3 min
2	Bhanu	Phrase and ISL modes (NEURAL NET badge once trained), word prediction fixing a misread letter, translate to Hindi and speak	~3 min
3	Manoj	Practice mode, Retrain button, `reports/evaluation.md` (before/after, curves, PCA/t-SNE), limitations	~3 min

2. What Every Member Must Know

2.1 The 30-second pitch

> "Vani-Setu turns sign language into text and speech using only a laptop webcam. MediaPipe finds 21 points on the hand in the browser, so video never leaves the device. We turn those points into 88 features and classify them with machine learning: an SVM reaches 96.9% on 26 fingerspelled letters and stays at about 97% when the other hand is used or the hand is tilted, where our first version fell to 64%. A Keras neural network handles whole signs with one or two hands, and word prediction fixes misread letters. The sentence is spoken and translated into 25+ languages, including Hindi."

2.2 Numbers to remember

Item	Value
Dataset	ASL Now Fingerspelling Landmarks (Hugging Face, MIT licence): 2,122 samples, 26 letters (the whole dataset)
Class sizes	Smallest C = 53 samples, largest Z = 155
Landmarks → features	21 points × (x, y, z) = 63 raw numbers → 88 features (scaled points + 10 fingertip distances + 15 joint angles); 181 for two hands
Split	70% train / 15% validation / 15% test, stratified, seed 42 (1,484 / 319 / 319)
Augmentation	Mirror + 2 tilted/jittered copies: 1,484 → 8,904 training samples (training split only)
Live letter model	SVM, RBF kernel, C = 3, γ = scale, class-weighted: 96.9% test, macro F1 0.972
Cross-validation	97.1% ± 1.6% (5 folds, augmenting inside each fold)
Before → after (same test set)	Normal 92.8% → 97.2%; other hand 64.3% → 97.2%; tilted 78.4% → 96.9%; hand farther away 72.1% → 97.2%
Other models (same data)	MLP 96.2%, Logistic Regression 95.9%, Random Forest 96.2%, KNN 95.0%, Decision Tree 90.9%
Keras MLP on letters	95.9% test (SVM 96.9% on the same samples); 21,338 parameters; best epoch 16
Speed	About 1 ms per frame for features + SVM; 17–20 ms browser round trip
Smoothing	Last 14 predictions, commit at 9 agreeing, confidence ≥ 0.70 (chosen on validation), 22-frame cool-down; a held sign types once
Word prediction	5,863 words; X-E-D → RED because R is sometimes read as X

2.3 What is ours and what we reused

Be open about this; examiners respect it. The project started from an open-source 2019 hackathon project (harshbg, MIT licence) that used an image CNN with hand-histogram segmentation. We replaced that approach completely and moved the old files to `legacy/` with credit: the landmark pipeline, the web app, the SVM and model comparison, the neural networks, the phrase and ISL modes, word prediction, translation, practice mode, retraining from the browser and all the evaluation are ours. MediaPipe (Google), the ASL Now dataset and the wordfreq word list are external, credited in the README.

2.4 Questions any member can be asked

Q. Why use landmarks instead of feeding images to a CNN?

A landmark vector has 63 numbers instead of 921,600 pixel values, so a small dataset is enough and the model runs in about a millisecond. Landmarks also ignore background and lighting, which broke the original image-CNN project.

Q. Is this a deep learning project?

Yes, in two ways. MediaPipe's hand model is a pre-trained neural network that we use as a feature extractor, which is transfer learning. We also trained our own Keras MLP. For letters it scored 95.9% against the SVM's 96.9%, so the SVM stays live for letters, and the MLP is used for whole signs with two hands. An LSTM for moving signs is our next step.

Q. Is 96.9% the real-world accuracy?

No. It is accuracy on a held-out part of the same dataset. We simulated harder conditions (mirrored, tilted, smaller hand) and the model stays at about 97%, but real users differ in more ways, so the app now records a separate new-signer test set that the trainer reports on its own.

Q. What are the limitations?

J and Z need motion, which a single frame cannot show; the phrase and ISL networks need our own recordings before they replace the rules; facial expressions are ignored. It is a communication aid, not a certified interpreter.

Q. How do you protect privacy?

Hand tracking runs in the browser. Only the hand points (63 numbers per hand) go to our server, never video. Translation sends only the text sentence.

3. Abdullah Naseem: Letter Recognition

Abdullah Naseem S24CSEU1193

Owns the path from a hand in front of the camera to a stable, confident letter.

3.1 Contribution

Status	Work	Where
BUILT	Version-2 features: palm-length scaling, 10 fingertip distances, 15 joint angles (88 numbers); version 1 kept for old models	`features.py`
BUILT	SVM training: 70/15/15 split, augmented training data, class weights, grid search for C and γ on validation, Platt probabilities	`train_classifier.py`
BUILT	Confidence threshold chosen on validation (0.70 keeps 98% of correct predictions) and saved in the model file	`train_classifier.py`
BUILT	Classify API: letter, probability and top 3, label always the highest probability; about 1 ms	`server.py`
BUILT	Camera loop: 14-frame vote, unsure frames vote "?", a held sign types once, confidence bar, response time	`index.html`
BUILT	Robustness test: mirrored, tilted and smaller hands on the test set	`train_classifier.py`

3.2 Concepts to explain

1. Hand landmarks with MediaPipe (transfer learning)

What it is	A pre-trained two-stage neural network from Google: a palm detector finds the hand, then a landmark model predicts 21 3D points (wrist, and 4 joints on each finger). It was trained on about 30,000 real hand images plus synthetic hands.
In our project	We use it as a ready-made feature extractor in the browser. Using a model trained on a large dataset for our own task is transfer learning. It shrinks a 640×480 colour frame (921,600 values) to 63 numbers.
Remember	Point 0 is the wrist; point 9 is the middle-finger knuckle; tips are 4, 8, 12, 16, 20.

2. Normalisation and invariance

What it is	Removing differences that are not part of the sign, so the model only sees hand shape.
In our project	Subtracting the wrist removes where the hand is in the frame (translation). Dividing by palm length removes how close the hand is to the camera (scale). Training on mirrored copies covers the other hand; angles and distances are the same for a mirrored hand anyway.
Formula	$p_i(new) = (p_i - p_0) / \lVert p_9 - p_0 \rVert$

3. Feature engineering: angles and distances

What it is	Creating new inputs from raw data using knowledge of the problem.
In our project	For each finger joint we compute the angle between the two bones that meet there (15 angles) and the distances between all pairs of fingertips (10). D and X differ only by a straight versus hooked index finger: their coordinates are close, but the joint angle is very different.
Formula	$\theta = \arccos((u \cdot v) / (\|u\| \, \|v\|))$, u and v = the two bone vectors at the joint
Remember	Same test set and SVM: 92.8% → 97.2% (with augmentation). With the other hand: 64.3% → 97.2%.

4. Feature scaling (standardisation)

What it is	Rescaling each feature to mean 0 and standard deviation 1.
In our project	SVM measures distances between samples, so a feature with a large range would dominate (angles are in radians, distances in palm lengths). StandardScaler is inside the pipeline, so its mean and standard deviation are learned from training data only, which avoids leaking test information.
Formula	$z = (x - \mu) / \sigma$

5. Support Vector Machine with RBF kernel

What it is	An SVM finds the boundary that leaves the widest margin between classes. Only the closest points, the support vectors, decide the boundary. The kernel trick lets it draw curved boundaries by measuring similarity instead of transforming the data explicitly.
In our project	With 26 letters, scikit-learn trains one SVM for every pair of letters (one-vs-one): 26 × 25 / 2 = 325 small classifiers that vote. C = 10 sets how strongly mistakes on training data are punished; γ sets how far one sample's influence reaches.
Formula	$K(x, x') = \exp(-\gamma \, \|x - x'\|^2)$
Remember	High C or high γ → tighter fit and risk of overfitting. Low values → smoother boundary and risk of underfitting.

6. Hyperparameter tuning with grid search

What it is	Trying every combination of settings and keeping the one with the best cross-validated score.
In our project	C in {3, 10, 30, 100} × γ in {scale, 0.01, 0.03} = 12 settings. Each is trained on the augmented training data and scored on the validation set (macro F1), never on the test set. Best: C = 3, γ = scale (validation macro F1 0.971). Scores from C = 3 to 100 differ by less than 1 point.
Remember	The tuned model scores 96.9% on test while an untuned C = 10 scores 97.2%: one sample. That is expected when you choose on validation, and it is the honest way.

7. Probability calibration (Platt scaling)	
What it is	An SVM outputs a distance from the boundary, not a probability. Platt scaling fits a sigmoid curve on those distances to turn them into probabilities.
In our project	This gives the confidence bar and the cut-off. Because the probabilities come from a separately fitted curve, the plain prediction and the highest-probability letter used to disagree (8 of 531 samples in our first version). The server now takes the letter from the probabilities, so they always match. The cut-off, 0.70, is the highest value that still keeps 98% of correct validation predictions; it rejects 30% of the errors.
Formula	$P(class \mid x) = 1 / (1 + exp(A \cdot f(x) + B))$

8. Temporal smoothing (majority vote over a sliding window)	
What it is	Combining several noisy predictions over time into one stable decision.
In our project	We keep the last 14 frame predictions (about half a second of video) and add a letter only when 9 of them agree. Frames below the 0.70 confidence cut-off vote "?". After a letter is added, the same letter is accepted again only after the hand changes, becomes unsure or leaves the frame, so holding a sign types it once (double letters: drop the hand briefly).
Remember	This is like an ensemble, but over time instead of over models.

3.3 One-minute viva script

"I built the letter-recognition stage. MediaPipe gives 21 hand points per frame. I turn them into 88 features that do not change with where the hand is or how big it looks: scaled points, fingertip distances and joint angles. A Support Vector Machine with an RBF kernel, tuned on a validation set, scores 96.9% on the test set, and with augmentation it stays at about 97% for the other hand or a tilted hand, where our first version dropped to 64%. Platt scaling gives a probability, a threshold chosen on validation rejects unsure frames, and a 14-frame vote turns flickering predictions into stable letters, in about 1 ms per frame."

3.4 Likely questions and answers

Q. Why an SVM and not a CNN?

Our input is 88 numbers, not an image, so a CNN has nothing spatial to learn from. On 2,122 samples the SVM was the best of six models on validation and scored 96.9% on test, ahead of our Keras MLP (95.9%). It classifies a frame in about 1 ms.

Q. What is the kernel trick?

It computes similarity as if the data had been mapped to a higher-dimensional space, without doing the mapping. In that space a straight boundary can separate classes that need a curved boundary in the original space.

Q. What happens if C is too large?

The model tries to classify every training point correctly, the margin shrinks and it overfits: high training accuracy, lower test accuracy.

Q. Why subtract the wrist?

So the same hand shape gives the same numbers anywhere in the frame. It makes the features translation invariant.

Q. Why fit the scaler only on training data?

If the scaler sees test data, the test set influences training. That is data leakage and gives an over-optimistic score.

Q. How many classifiers does the SVM train for 26 letters?

325, one for every pair of letters (one-vs-one). The letter with the most votes wins. The trained model keeps 2,684 support vectors.

Q. Where does the confidence number come from?

Platt scaling: a sigmoid fitted to the SVM's decision values during training, using internal cross-validation. Frames below 0.70 count as unsure.

Q. Why do you need the 14-frame vote?

Single frames flicker between similar letters while the hand moves. Requiring 9 of 14 agreeing frames removes that noise at the cost of about half a second of delay.

Q. What if a left-handed person signs?

We train on mirrored copies of every sample, so a left hand looks familiar. On the mirrored test set the new model scores 97.2%; the original scored 64.3%. Angles and distances are also the same for a mirrored hand.

Q. Which letters are hardest and why?

In the first version P was read as Z and D and R as X: they share an index-finger shape. Joint angles removed those. Now the 10 test errors are all single cases (U/V, U/R, Z/X, Z/J); J and Z still need motion, which is Bhanu's LSTM.

4. Bhanu Satwik: Words, Phrases and Language

Bhanu Satwik S24CSEU0898

Owns everything above single letters: phrases, ISL signs, moving signs, word prediction, translation and speech.

4.1 Contribution

Status	Work	Where
BUILT	Keras MLP trainer for letters, ASL phrases and ISL signs, with loss and accuracy curves; letters: 95.9% vs SVM 96.9%	`nn/train_mlp.py`
BUILT	Exported weights run with NumPy on the server, so the website needs no TensorFlow	`nn/numpy_mlp.py`
BUILT	Two-hand tracking: both hands recorded with their Left/Right label; 181 two-hand features	`index.html`, `features.py`
NEEDS DATA	Phrase and ISL networks replace the rules (badge NEURAL NET) once ~25 samples per sign are recorded and trained	`nn/train_mlp.py`
BUILT	Noisy-channel word prediction using word frequencies and the letter model's confusion counts	`suggest.py`
BUILT	Translation into 25+ languages with an offline Hindi fallback; speech; login	`server.py`
ADDING	LSTM sequence model over 30 frames for J, Z and moving signs	`nn/`

4.2 Concepts to explain

1. Rule-based versus learned classification

What it is	A rule-based system uses conditions written by a person; a learned model finds the decision rules from labelled examples.
In our project	Today, Phrase mode checks which fingers are up (fingertip above its middle joint) and fingertip distances divided by palm length, for example "thumb, index and pinky up = I Love You". Rules are quick to write but break for new people and cannot scale to many signs, so a trained network takes over as soon as one exists for that screen, and the rules stay only as a fallback.

2. Multi-layer perceptron (MLP)

What it is	A neural network of fully connected layers. Each neuron computes a weighted sum of its inputs plus a bias and passes it through an activation function.
In our project	Our network: 181 inputs for two hands (88 for letters) → Dense 128 (ReLU) → Dropout 0.3 → Dense 64 (ReLU) → Dropout 0.3 → Dense N (softmax), where N is the number of signs. For letters that is 21,338 weights. ReLU passes positive values and zeroes negatives; it avoids the vanishing gradients of sigmoid.
Formula	$a = ReLU(w \cdot x + b)$, $ReLU(z) = max(0, z)$

3. Softmax and cross-entropy loss

What it is	Softmax turns the last layer's scores into probabilities that add up to 1. Cross-entropy measures how far those probabilities are from the true label.
In our project	Softmax gives the probability of each phrase. The loss is low when the network gives high probability to the correct phrase and very high when it is confidently wrong.
Formula	$softmax(z)_k = exp(z_k) / \Sigma_j exp(z_j)$ $L = -\Sigma_k y_k \cdot log(\hat{y}_k)$

4. Backpropagation and the Adam optimiser

What it is	Backpropagation uses the chain rule to find how much each weight contributed to the loss. Gradient descent then moves each weight a small step against its gradient.
In our project	We train with Adam (learning rate 0.001), which adapts the step size for each weight and adds momentum, so it converges faster than plain gradient descent.
Formula	$w \leftarrow w - \eta \cdot \partial L / \partial w$

5. Preventing overfitting: dropout and early stopping

What it is	Dropout randomly switches off a fraction of neurons in each training step, so the network cannot rely on a few neurons. Early stopping ends training when validation loss stops improving.
In our project	Phrase datasets are small (a few hundred samples per sign), so the network can easily memorise them. We use dropout 0.3 and stop when validation loss has not improved for 10 epochs, keeping the best weights. On letters it stopped after 26 epochs and kept epoch 16. Validation accuracy starts above training accuracy because dropout is switched off when validating.
Remember	On 2,122 letter samples the MLP scored 95.9% against the SVM's 96.9% on the same test set: neural networks usually need more data to win.

6. Sequence models: RNN and LSTM

What it is	A recurrent network reads inputs one time step at a time and keeps a hidden state that remembers earlier steps. An LSTM adds gates (forget, input, output) that decide what to keep, which avoids the vanishing-gradient problem of plain RNNs.
In our project	J and Z, and most word signs, are defined by movement. Our model reads 30 frames × 63 features (about one second) through an LSTM layer of 64 units, then a softmax layer. A single-frame model cannot tell P from Z; a sequence model sees the drawing motion.

7. Noisy-channel model for word prediction (Bayes' rule)

What it is	Treat recognised letters as a noisy version of the word the user meant, and pick the word that best explains them.
In our project	P(word) comes from the frequencies of 5,863 English words (wordfreq). P(observed letter | intended letter) comes from the letter model's confusion counts on held-out data, smoothed so unseen mistakes stay possible. In the app, X-E-D suggests RED, and THAMK suggests THANK; corrected suggestions have a dashed border. The old prefix-only match could not fix a misread letter.
Formula	$\hat{w} = \arg\max_w P(w) \times \prod_i P(\text{observed}_i \mid w_i)$

8. Neural machine translation (used, not trained)

What it is	Modern translation uses encoder-decoder transformer networks trained on huge parallel text corpora.
In our project	We call Google Translate through the deep-translator library for 25+ languages. We do not train this model; if the service fails, a small offline English-to-Hindi dictionary keeps Hindi working. Speech uses the browser's built-in text-to-speech.

9. Deploying a network without its framework

What it is	A trained network is just its weights and the arithmetic of the forward pass.
In our project	After Keras training we save the scaler and each layer's weights to a small .npz file. The server repeats the forward pass in NumPy (matrix multiply, ReLU, softmax), and we check that NumPy and Keras give the same validation accuracy. This keeps TensorFlow (about 500 MB) off the free web server.
Formula	$h = \max(0, x W1 + b1)$ $h2 = \max(0, h W2 + b2)$ $p = \text{softmax}(h2 W3 + b3)$

4.3 One-minute viva script

"I handle everything above single letters. Our first version recognised phrases with hand-written finger rules. I built a Keras MLP with ReLU, dropout, softmax, cross-entropy and early stopping that learns signs from recorded samples, and the app now tracks two hands so real ISL signs fit. The trained weights run with NumPy on the server. On letters the MLP reached 95.9%, close to the SVM. I also use Bayes' rule for word prediction, so X-E-D still suggests RED, and the sentence is translated into 25+ languages and spoken. Next is an LSTM for moving signs like J and Z."

4.4 Likely questions and answers

Q. Why replace the rules with machine learning?
Rules were written for one person's hand and need a new condition for every sign. A trained model learns from many people's samples and adding a sign only needs more data, not new code.

Q. Why an MLP for phrases when the SVM was better for letters?
The gap on letters was one point (95.9% vs 96.9%). Phrases need two hands (181 features) and will gain new signs over time; a network handles that well and leads on to the LSTM. Once phrase data is recorded we will compare both and keep the better one.

Q. What does ReLU do and why not sigmoid?
ReLU outputs max(0, z). Its gradient is 1 for positive values, so gradients do not shrink across layers the way they do with sigmoid.

Q. What is dropout?
During training, a random 30% of neurons are switched off in each step, forcing the network to spread what it learns. At prediction time all neurons are used.

Q. What does softmax output?
One probability per class, all adding to 1. The highest probability is the prediction, and its value is the confidence.

Q. Why an LSTM for J and Z?
Their meaning is in the movement, so the model needs the order of frames. An LSTM keeps memory across the 30 frames; an MLP or SVM sees only one frame.

Q. How does the two-hand model handle a frame with only one hand?
Each hand has a slot (left, right) chosen from MediaPipe's handedness label. A missing hand's 88 values are zeros and two presence flags say which hands are visible, so the input stays 181 numbers. Three more numbers give the second wrist's position relative to the first.

Q. Give an example of the word prediction.
If the user signs R-E-D but the model reads X-E-D, the confusion matrix says R is often read as X, and RED is a common word, so RED is ranked first.

Q. Is the ISL mode real Indian Sign Language?
The built-in rules are a simplified set of one-handed classroom signs, not a certified ISL vocabulary. With two-hand tracking and the ISL network, real two-handed ISL signs can be recorded and learned.

Q. How does the server run the network without TensorFlow?
We export the weights to a .npz file and repeat the forward pass in NumPy: matrix multiply, ReLU, softmax. The NumPy version gives the same validation accuracy as Keras (97.2% on letters).

Q. What happens to translation without internet?
Hindi still works through our offline dictionary; other languages need the translation service.

5. Manoj Kumar K: Data and Evaluation

Manoj Kumar K S24CSEU1239

Owns the data every model learns from and the evidence that shows how well each model works.

5.1 Contribution

Status	Work	Where
BUILT	Raw-landmark storage for every sample with signer, hand and split; ASL Now converted to one CSV	`data.py`, `build_dataset.py`
BUILT	Record panel with signer name and Test set box; recording and retraining only on localhost	`index.html`, `server.py`
BUILT	Augmentation (mirror, ±15° tilt, jitter) on training data only: 1,484 → 8,904 samples	`augment.py`
BUILT	70/15/15 split, class weighting, six-model comparison on identical data	`train_classifier.py`, `compare_models.py`
BUILT	Evaluation report: before/after, CV, learning and validation curves, PCA/t-SNE, speed, per-signer results	`evaluate.py`
NEEDS DATA	New-signer test set: 4–5 classmates not in training, all 26 letters, Test set box ticked	`dataset/recordings/`

5.2 Concepts to explain

1. Supervised learning data and class imbalance

What it is	Each sample is a feature vector with a correct label. If some classes have far more samples, a model can favour them.
In our project	Our 2,122 samples range from 53 (C) to 155 (Z). We report macro F1, which weights every letter equally, and use class_weight="balanced", which gives rare classes a larger weight in the loss.

2. Data augmentation for landmarks

What it is	Creating realistic new training samples by slightly changing existing ones.
In our project	Each training sample is mirrored (other hand), then the original and mirror each get 2 copies tilted by up to ±15° with small random noise on every point (jitter): 1,484 → 8,904 samples. It is applied only to the training split, so test results stay honest. On normal test data it changes little, but on the mirrored test set it lifts the original 63-feature model from 64.3% to 94.4%.

3. Train, validation and test sets; data leakage

What it is	Training data fits the model, validation data chooses settings, and test data is used once at the end to report the final score. Leakage is any way test information reaches training.
In our project	We use 70/15/15 (1,484 / 319 / 319). C and γ, the model choice and the confidence threshold are all chosen on validation; test is used once. Leakage risks here: fitting the scaler on all data, augmenting before splitting (near-copies on both sides), and the same signer in train and test.

4. Stratified k-fold cross-validation

What it is	Split the training data into k parts, train on k−1 and test on the remaining one, repeat k times and average. Stratified means every part keeps the same letter proportions.
In our project	We use k = 5 on train + validation, augmenting inside each fold so no copy of a sample crosses folds. The SVM scores 97.1% ± 1.6% (folds 98.6, 98.1, 98.3, 95.3, 95.0).
Formula	$CV\ score = (1/k)\ \Sigma_i\ accuracy_i$

5. Model comparison and the bias–variance trade-off

What it is	Bias is error from a model that is too simple; variance is error from a model that changes too much with the training data.
In our project	Same data, features and split for six models (test accuracy): SVM 97.2%, MLP 96.2%, Random Forest 96.2%, Logistic Regression 95.9%, KNN 95.0%, Decision Tree 90.9%. One tree has high variance (it memorises noise); Random Forest averages 200 trees (bagging) to fix that but takes 51 ms per prediction. Logistic Regression shows the letters are almost linearly separable; the SVM's curved margin adds the last points. Better features lifted every model: the tree went from 74.6% to 90.9%.

6. Evaluation metrics

What it is	Precision: of the predictions for a letter, how many were right. Recall: of the real samples of a letter, how many were found. F1 is their harmonic mean. Macro F1 averages F1 over letters equally.
In our project	Accuracy alone hides weak letters. Our test macro F1 is 0.972. U has the lowest recall (0.875): two of its 16 test samples were read as R and V.
Formula	$Precision = TP / (TP + FP)$ $Recall = TP / (TP + FN)$ $F1 = 2PR / (P + R)$

7. Confusion matrix and error analysis

What it is	A table of actual versus predicted classes. The diagonal is correct; everything else is a specific mistake.
In our project	In the first version 26 of 531 test samples were wrong, mostly P → Z, D → X and R → X; that is why we added joint angles. Now 10 of 319 are wrong, each a single case (U/R, U/V, Z/X, Z/J…). The same counts feed Bhanu's word prediction.

8. Learning curves and validation curves

What it is	A learning curve plots training and validation score as the training set grows. A validation curve plots them as one hyperparameter changes.
In our project	Learning curve: validation accuracy rises from 83.7% with 148 samples to 96.2% with 890, then flattens (96.5% at 1,484). More samples like these add little; samples from new people matter more. Validation curve: C = 0.03 underfits (82.8%); from C = 3 validation stays at 96.9% while training reaches 100%, a growing gap.

9. Dimensionality reduction for visualisation (PCA, t-SNE)

What it is	PCA projects data onto the directions of largest variance. t-SNE places points so that neighbours stay close.
In our project	PCA's first two components explain 54% of the variance and the letters overlap heavily, which is why a linear 2-D view is not enough. t-SNE shows clear clusters per letter, some split in two (the dataset has both hand orientations).

10. Domain shift and real-world testing

What it is	A model can score well on data like its training set and worse on data from a different setting.
In our project	The dataset does not record who signed each sample, so the same person may be in train and test. We simulated shifts (mirrored, tilted, farther hand): the original model fell to 64–78%, the new one stays at about 97%. The new-signer test set measures real people the model has never seen (target: at least 85%). Speed: about 1 ms per frame on the server, 17–20 ms round trip in the browser.

5.3 One-minute viva script

> "I own the data and the evaluation. Every sample is stored as raw landmarks with the signer's name, and a Test set box keeps new people out of training. I split the data 70/15/15, augment only the training part, and compared six models on identical data: the SVM won. Cross-validation gives 97.1% ± 1.6%. My before/after test shows the first model fell to 64% with the other hand and the new one stays at 97%. The learning curve flattens, so the next gain comes from new signers, which is what our new-signer test set measures."

5.4 Likely questions and answers

Q. Why is accuracy not enough?

With uneven classes, a model can be accurate overall and still fail on rare letters. Per-class precision, recall and macro F1 show those failures.

Q. What is the difference between validation and test data?

Validation data is used repeatedly to choose models and settings. Test data is touched once, at the end, so the reported score is not biased by those choices.

Q. Where could data leakage happen in your project?

Fitting the scaler on all data, augmenting before splitting or across CV folds, and the same signer in train and test. We prevent the first two with a pipeline and split-then-augment (inside each fold for CV), and the new-signer test set addresses the third.

Q. Why did the Decision Tree do worst?
A single deep tree memorises the training data (high variance). Random Forest averages 200 trees to fix this and scored 5.3 points higher (96.2% vs 90.9%).

Q. Why stratified splitting?
So each split has the same proportion of every letter; otherwise a rare letter like C might be almost missing from the test set.

Q. What does a 5-fold CV score of 97.1% ± 1.6% mean?
The average accuracy over five different train/validation splits was 97.1%, and it varied between 95.0% and 98.6%, so a single test score of about 97% is believable.

Q. Why augment only the training data?
The test set must represent real, unchanged samples. Augmenting it would measure performance on our own synthetic data.

Q. What does your learning curve tell you?
Validation accuracy flattens at about 96.5% after roughly 900 training samples, so more samples of the same kind help little. Samples from new people are what will help.

Q. What accuracy do you expect on new signers?
Lower than 95%, because of different hands and cameras. Our target is at least 85%, and the gap tells us how much augmentation and normalisation help.

Q. How does practice mode relate to AI?
It uses the live classifier to check the user's sign against a prompted letter and scores it. It skips J and Z, which a single-frame model cannot judge fairly.

6. Viva Day Checklist

- Open before the viva: the running app, `reports/evaluation.md`, `reports/model_comparison.md`, `reports/nn_letter.md`, `features.py` and `train_classifier.py`.
- Speak in order: Abdullah, then Bhanu, then Manoj, about three minutes each.
- If asked about another member's part, give the one-line answer from Section 2, then let the owner explain.
- Separate clearly what is built from what is not. Say "we are implementing" for ADDING items and "built, waiting for our recordings" for NEEDS DATA items.
- Quote numbers exactly: 2,122 samples, 26 letters, 88 features, 96.9% test, 97.1% ± 1.6% CV, 64% → 97% with the other hand, MLP 95.9%.
- Credit the starting project, MediaPipe and the dataset if asked what is original.